

Chapter 4: Open Table Formats

Preparation For AI: From Raw Data To Reliable Models

Alexander D'hoore

Open Table Formats

Chapter 3 gave us **efficient Parquet files**.

Chapter 4 asks:

When the dataset changes, who keeps track of table state?

The answer is an **open table format**.



Files Are Not Table State

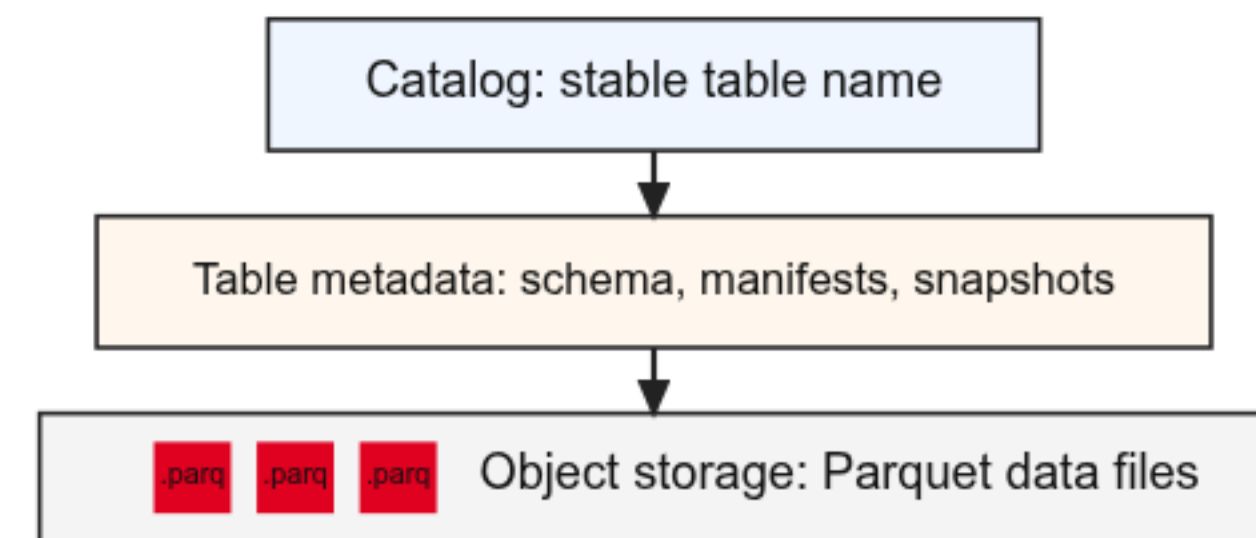
A folder of Parquet files can be efficient.

But a folder does not reliably tell you:

- which files are current
- which schema is active
- whether a write finished
- which version trained a model

Naming conventions help. They are not enough forever.

A table format adds state above files



The Missing Word Is State

AI data is rarely static.

New batches arrive.

Bad data gets corrected.

Schemas evolve.

Models need reproducible training evidence.

The table needs a memory.



Parquet Is Not Iceberg

Parquet is the **file format**.

It controls how columns and rows are encoded inside files.

Iceberg is the **table format**.

It controls which files form a table over time.



What Iceberg Adds

An Iceberg table records:

- data files
- metadata JSON files
- manifests
- snapshots
- schema and partition specs
- table properties

You do not query a random folder anymore.

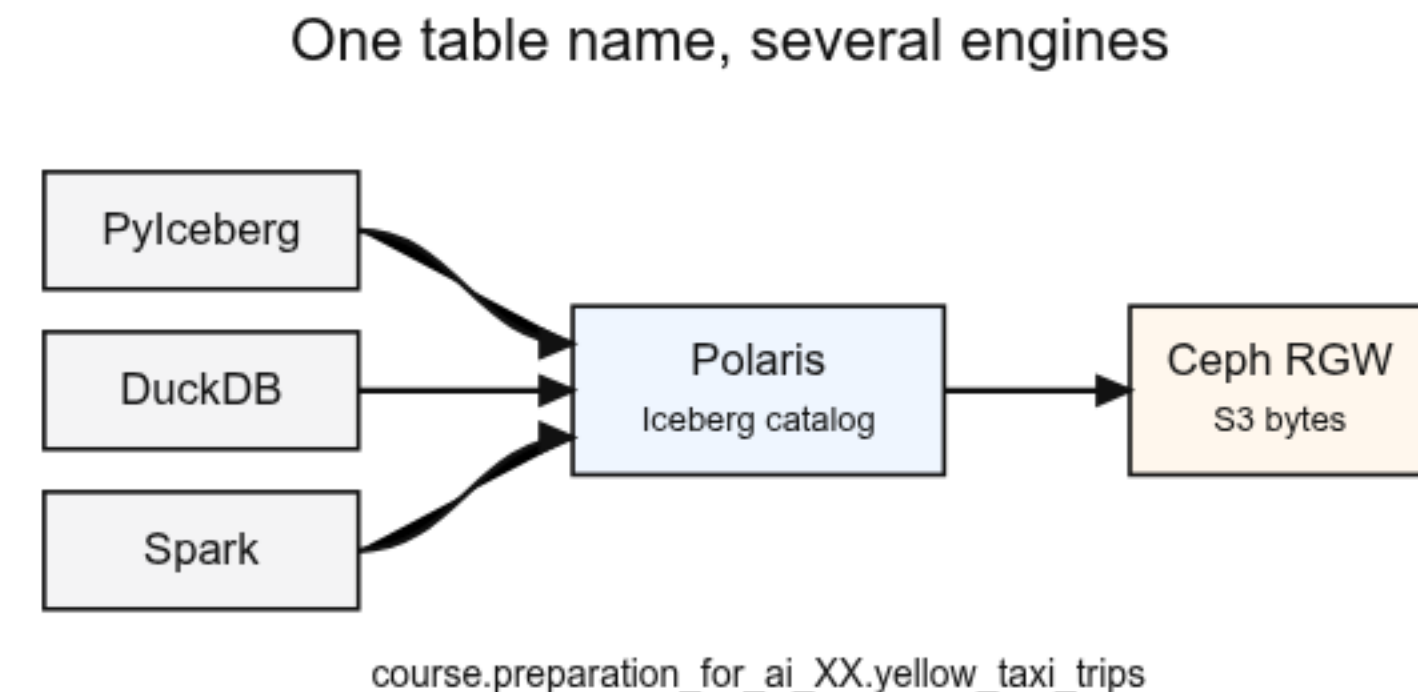


Three Different Jobs

Keep these roles separate:

- Parquet stores rows and columns efficiently
- Iceberg records table state over time
- Polaris lets tools find the table by name

The bytes, the history, and the name are separate concerns.



The Catalog

A catalog lets tools use a stable table name:

```
select *  
from  
course.preparation_for_ai_01.yellow_taxi_trips;
```

The catalog points to the **current metadata**.

Without a catalog, users pass around paths.



Polaris In This Course

Polaris is the shared Iceberg catalog service.

Ceph RGW stores the objects.

Your participant namespace owns your tables.

Pylceberg, DuckDB, and Spark can resolve the same table name.



Why This Is Lakehouse-Like

Object storage keeps data lake flexibility.

The table format adds:

- named tables
- schema management
- snapshots
- transactional commits
- multi-engine access

That is the lakehouse idea in miniature.

ACID, Practically

For this chapter, ACID means:

a write becomes a valid new table state,
or it does not become visible as success

Readers should not see a half-written table.



Appends Create New Snapshots

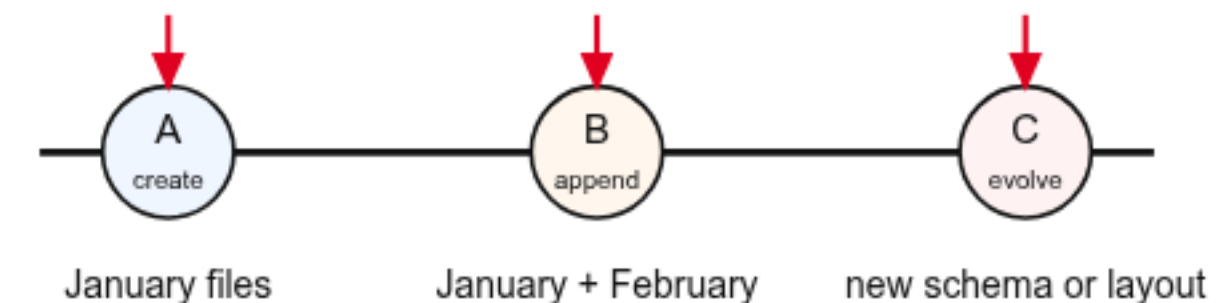
An append does not edit old Parquet files in place.

Usually it:

- writes new data files
- writes new metadata
- commits a new snapshot
- moves the table pointer forward

The table name stays the same.

Snapshots record table state over time



A model can point to one exact snapshot ID.

Snapshot IDs Are Evidence

For AI, this is weak:

`trained on the taxi table`

This is stronger:

`trained on snapshot 123456789`

The dataset version becomes part of the model record.



Metadata Tables

Iceberg metadata can be queried.

For snapshots:

```
select committed_at, snapshot_id, operation  
from  
course.preparation_for_ai_01.yellow_taxi_trips.  
order by committed_at;
```

The table has rows and history.

Schema Evolution

Schema changes should be **recorded table operations**.

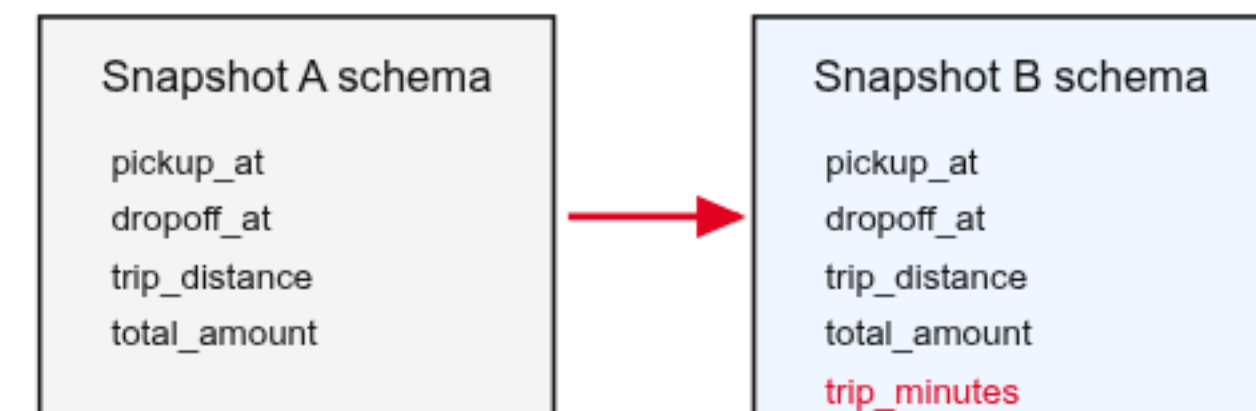
Adding a nullable feature is usually manageable.

Changing a target label is serious.

Renaming join keys can break downstream code.

The format gives mechanics. Engineers still need discipline.

Schema change becomes a table operation



Older snapshots remain interpretable.

Open Table Formats

The main names to recognize:

Iceberg, Delta Lake, and Hudi all add table structure above files.

This course uses Iceberg as the concrete example.



Lab Orientation

The lab uses sampled public NYC Yellow Taxi records.

One table, three tools:

- PyIceberg creates January
- DuckDB queries the table
- Spark appends February
- DuckDB verifies the new snapshot

The table name stays stable. The table state changes.



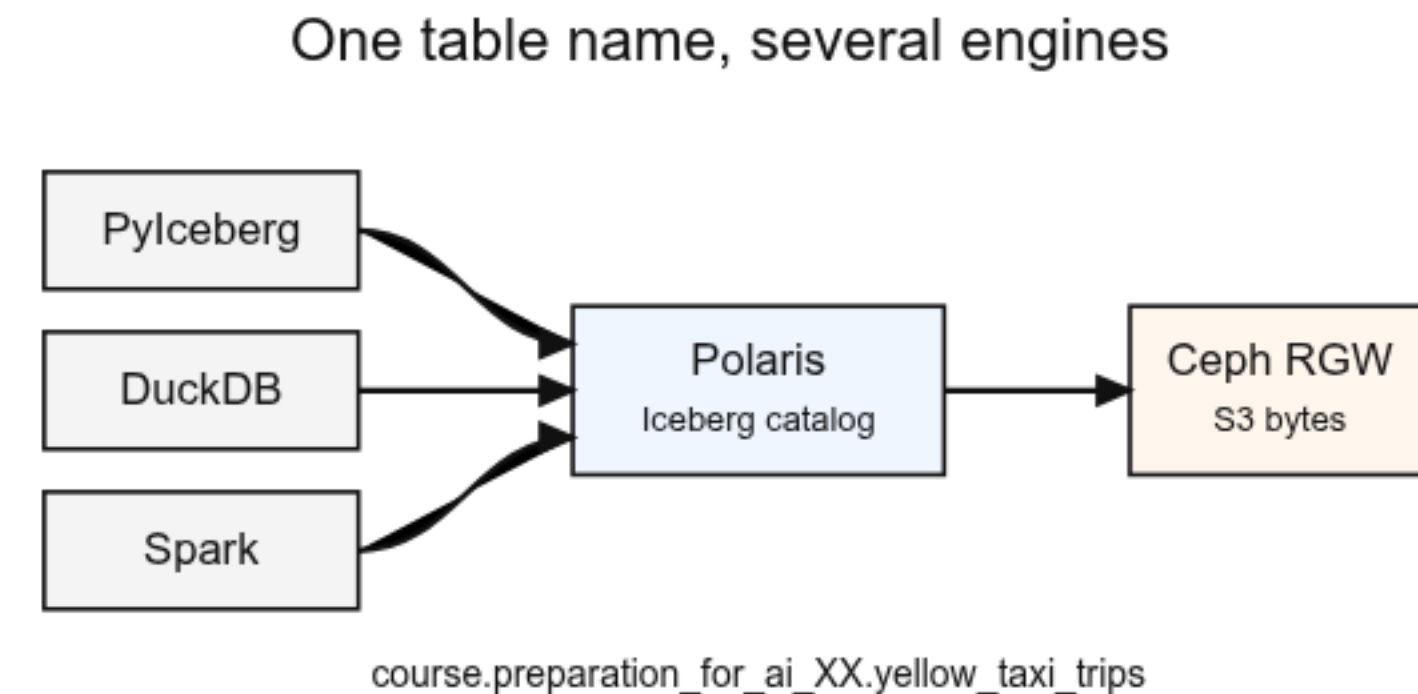
Interoperability Is The Point

The same table is touched by:

- PyIceberg from Python
- DuckDB through SQL
- Spark on Kubernetes

They coordinate through the catalog and metadata.

The data remains in object storage.



What To Notice In The Lab

Look for:

- one stable table name
- changing snapshot IDs
- row count after the append
- DuckDB and Spark seeing the same state
- Parquet files still underneath

Debug the table name and snapshot first.

Open Table Formats

Plain Parquet gives efficient files.

Iceberg gives **table state**.

Polaris gives table identity.

Snapshots give **reproducible versions**.

Multiple engines can work together without owning the data.

Next: use manifests for multimedia assets.